

PORTFOLIOGENIE

An AI-Powered Developer Portfolio Compiler & Static Exporter

GRADUATION PROJECT PRESENTATION

Digital Egypt Pioneers Initiative (DEPI) – Front-End React Track | Ministry of Communications & IT

Prepared by Project Team Members:

1. Elsayed Mohamed Elsayed (Team Lead)
2. Mohamed Sherif Abdelrahim
3. Mostafa Mohamed Taha
4. Youssef Shady Gabr
5. Youssef Mohamed Sadeq
6. Ahmed Mostafa Riad

What is PortfolioGenie?

- ◆ A client-side web application designed to help developers build a professional portfolio website in under 5 minutes.
- ◆ Allows users to input their GitHub username to automatically fetch public repositories, language statistics, and bio details.
- ◆ Features a visual workspace with a split-screen layout: editable forms on the left and a live preview canvas on the right.
- ◆ Supports dynamic custom additions including experience and education timelines.

The Core Deliverables

- ◆ Dynamic compiler engine that bundles all visual templates, CSS structures, and script elements together.
- ◆ Generates a single standalone HTML file with zero external dependencies.
- ◆ The output file is 100% portable, runs offline on any desktop or mobile browser, and can be hosted for free on GitHub Pages.
- ◆ Optimized frontend performance without server maintenance costs.

The Developer's Friction

- ◆ Junior web developers and students waste 16+ hours writing boilerplate CSS styles for their resumes instead of coding features.
- ◆ Static PDF files fail to showcase interactive layouts, responsive structures, or continuous integration/deployment details.
- ◆ Managing backend database setups and cloud hosting configurations is highly complex and financially costly for job seekers.

Our Visual Solution

- ◆ Eliminates builder's block by replacing code configuration steps with a simple, automated GitHub API importer wizard.
- ◆ Provides visual live preview checks so users see their structural portfolio changes instantly before downloading.
- ◆ Offers a serverless, database-free compiler tool that operates client-side to guarantee user privacy.

Phase 1: Setup & Fetching

- ◆ User lands on the interactive onboarding page and inputs their GitHub username.
- ◆ The application calls public API endpoints to retrieve user profile data, public repositories, and language usage stats.
- ◆ React Context updates, and the Builder Dashboard parses the API results into editable state forms.

Phase 2: Customization & Export

- ◆ User refines details across 3 dashboard tabs: Profile data, Projects customize, and Timeline entries.
- ◆ The Live Preview Canvas renders layout adjustments in real-time.
- ◆ User triggers the export script to download a compiled standalone HTML bundle.

Frameworks & Library Core

- ◆ React 18: Used for structured component composition, global context states, and dynamic state reducer management.
- ◆ React Router: Controls page transition routes between LandingPage onboarding and the dashboard Builder workspace.
- ◆ Framer Motion: Delivers premium micro-animations and smooth transition entrances for all tab elements.
- ◆ Vanilla CSS Modules: Implements a clean frontend style tokens system using variables for layout portability.

APIs & Web Technologies

- ◆ GitHub REST API: Feeds profile metadata and repository statistics asynchronously without credentials.
- ◆ LocalStorage API: Auto-saves current session data automatically to protect against tab closures.
- ◆ FileReader API: Reads uploaded avatar images as base64 data URLs to avoid server hosting.
- ◆ Vite Tooling: Modern build system ensuring rapid Hot Module Replacement (HMR) and optimized loading.

Leadership, Design & Documentation Core

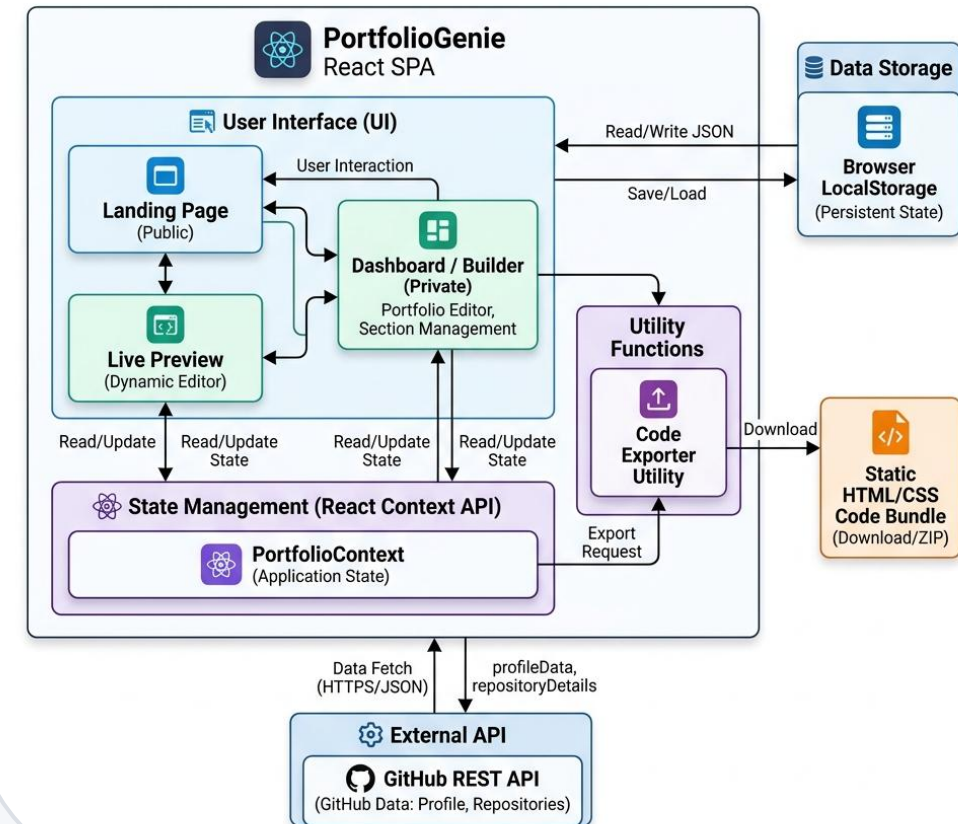
- ◆ Elsayed Mohamed Farag (Team Lead): Developed the Landing Page completely, designed the full UI/UX, compiled the entire project documentation, and designed the final presentation slides.
- ◆ Mohamed Sherif Abdelrahim (Developer): Developed the Profile Tab inputs and the real-time Live Preview Canvas layout.
- ◆ Mostafa Mohamed Taha (Developer): Developed the async GitHub API integration and languages cache hook.

Customization & Compilation

- ◆ Youssef Shady Gabr (Developer): Developed the Projects Tab custom cards manager and manual project builder UI.
- ◆ Youssef Mohamed Sadeq (Developer): Developed the Experience & Education Timeline builders with add/delete forms.
- ◆ Ahmed Mostafa Riad (Developer): Developed the AI biography enhancer, LocalStorage sync reducer, and HTML compiler.

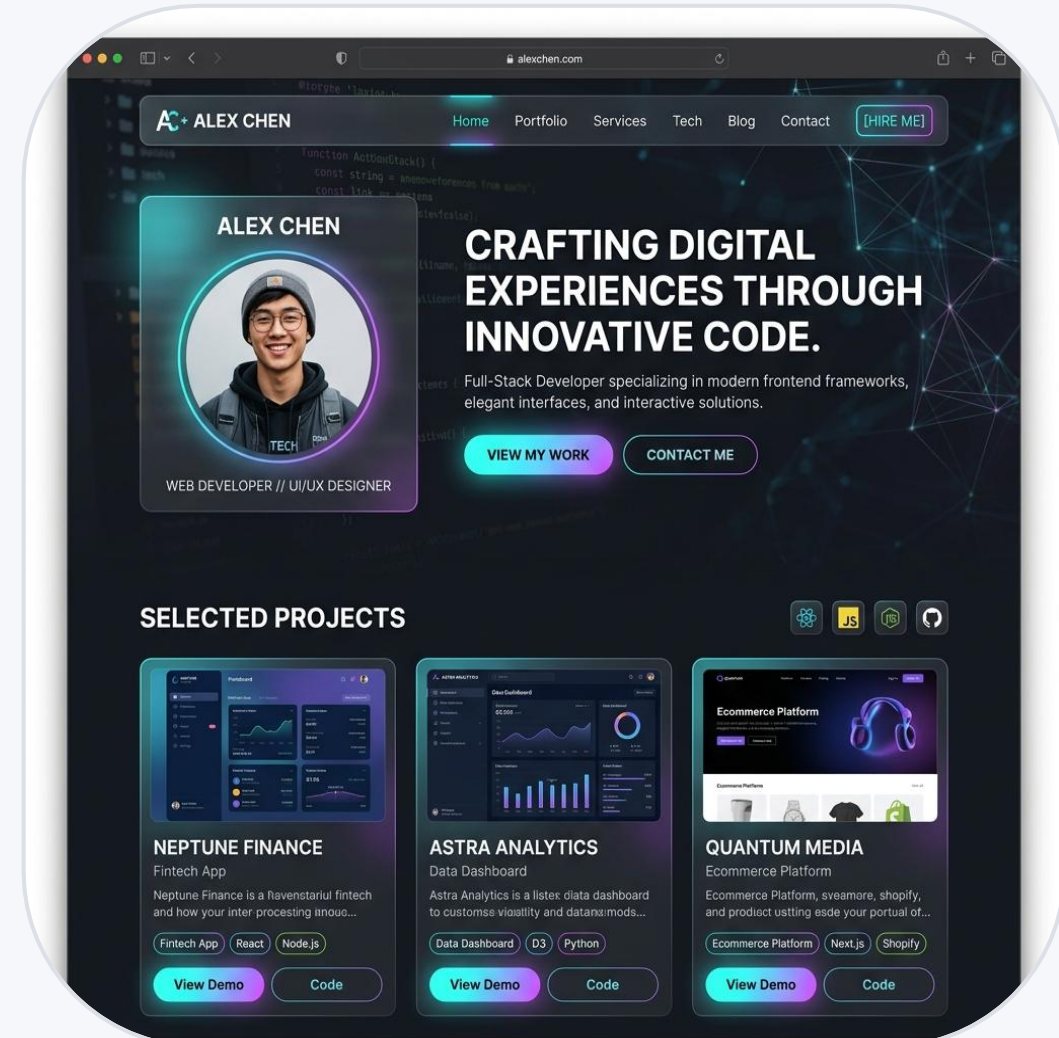
High-Level Architecture Flow

- ◆ Model-View-Controller (MVC) pattern implemented completely client-side in the browser.
- ◆ View Layer: Form editors (Profile, Projects, Timeline) and the Live Preview Canvas react to context updates.
- ◆ Controller Layer: React hooks, reducer actions, and context API manage state transitions and database resets.
- ◆ Model Layer: Structured JSON schema synced dynamically to LocalStorage and exported into HTML.



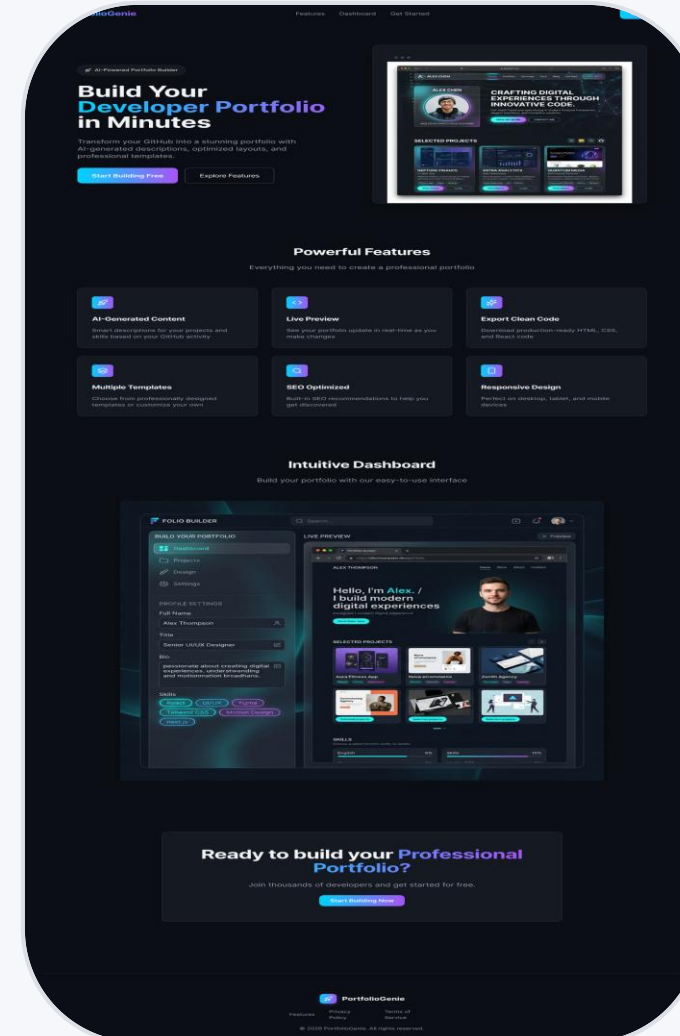
The Compiled Portfolio File

- ◆ Zero External Files: Inlines all styling sheets, typography setups, SVG icons, and interactive scripts.
- ◆ Scroll Listener Script: Auto-updates active indicators inside the navigation header on scroll.
- ◆ Glow Border Gradient Effect: Incorporates a modern, animated gradient border glow (CSS linear-gradient border animation).
- ◆ Responsive Design: Implements CSS media queries so the compiled layout renders perfectly on all screen widths.



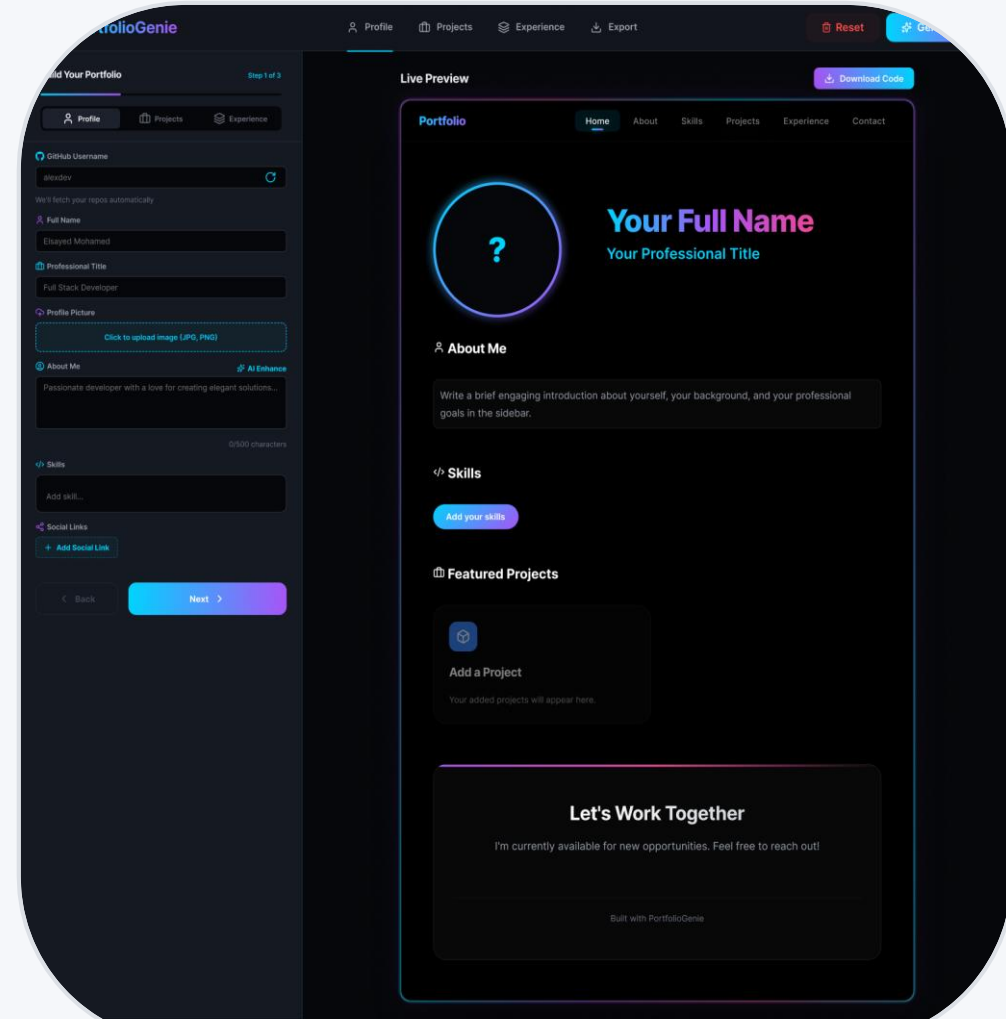
What We Developed (LandingPage.jsx)

- ◆ Onboarding Navigation Bar: Provides smooth-scroll links to features and quick access to the builder.
- ◆ Animated Hero Area: Implements entrance animations using Framer Motion (initial Y offset: 30 to 0).
- ◆ Feature Highlights Grid: Showcases interactive cards displaying the system's capabilities.
- ◆ Modern Styling Design: Implements custom CSS variables, background gradient highlights, and glassmorphism elements.



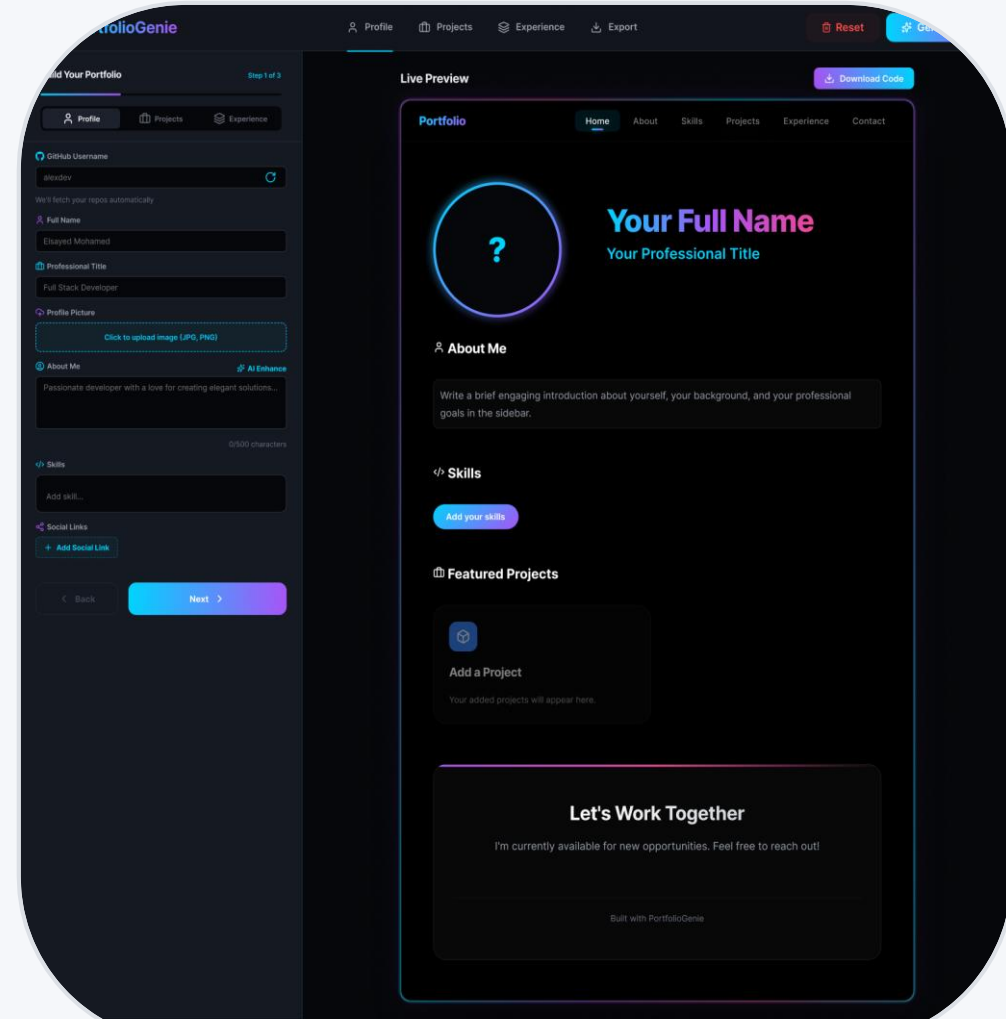
What I Built (ProfileTab.jsx)

- ◆ Interactive Inputs: Dynamic form fields for Full Name, Professional Title, and social links dropdowns.
- ◆ Platform Link Entries: Manages an array of social links (Gmail, Twitter, WhatsApp, GitHub, LinkedIn).
- ◆ Input Animations: Smooth transition styling on focus using Vanilla CSS and Framer Motion context.
- ◆ Context Dispatcher: Immediately pushes user form inputs into the global portfolioData state.



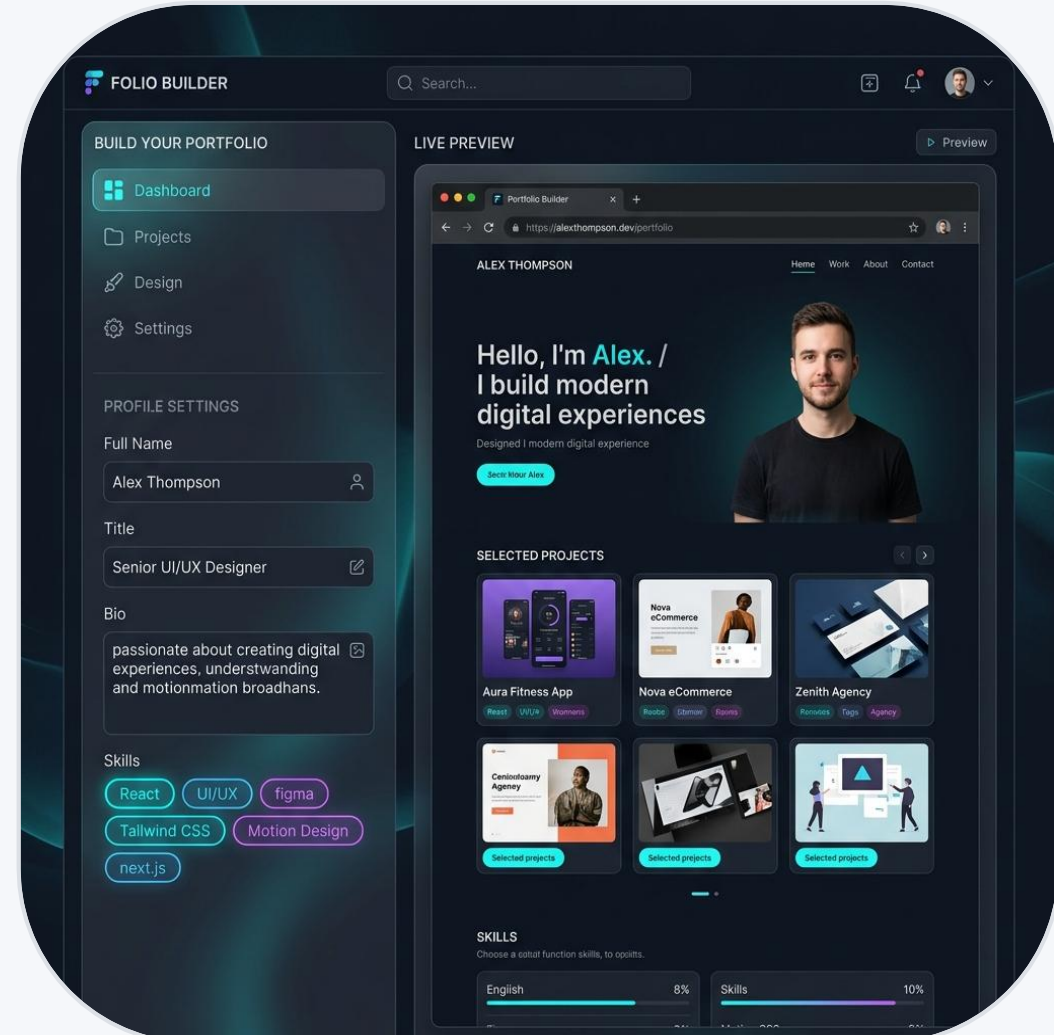
Profile Assets & Tags Management

- ◆ FileReader Upload: Converts profile avatar file uploads locally into Base64 data URLs.
- ◆ Skills Tags Editor: Keyboard listener (Enter) that processes input into removable skills badges.
- ◆ Duplicate Prevention: Integrates array matching checks before appending tags to avoid double cards.
- ◆ Bio Character Limiter: Implements a live character indicator (/500 characters) for the Bio editor.



Real-Time Preview Canvas

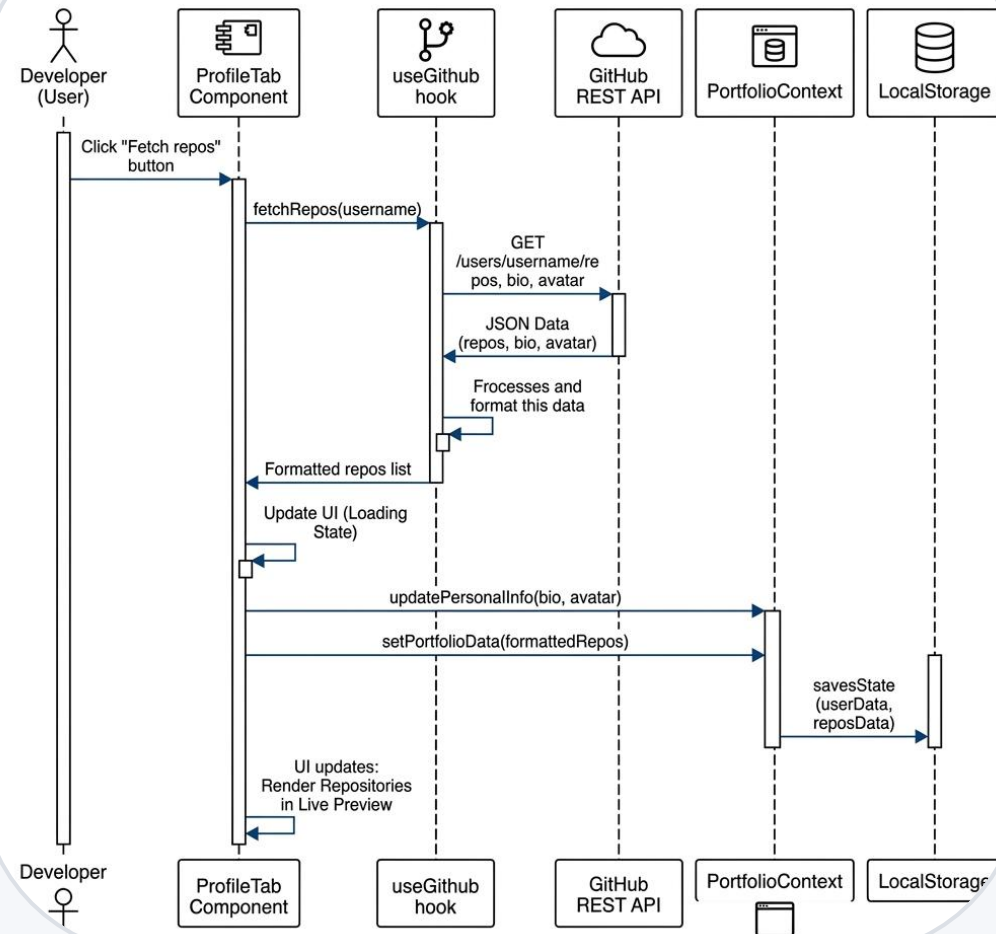
- ◆ State Binding: Connects to the global state provider to automatically re-render components on changes.
- ◆ Scroll Sync Logic: Automatically scrolls the preview pane to the active section when user changes text.
- ◆ Viewport Toggle Hooks: Incorporates viewport toggle parameters (isMobileView) for mobile/desktop layout tests.
- ◆ Identical Output: Guarantees 100% look-and-feel alignment with the final exported standalone HTML.



What We Built (useGithub Hook)

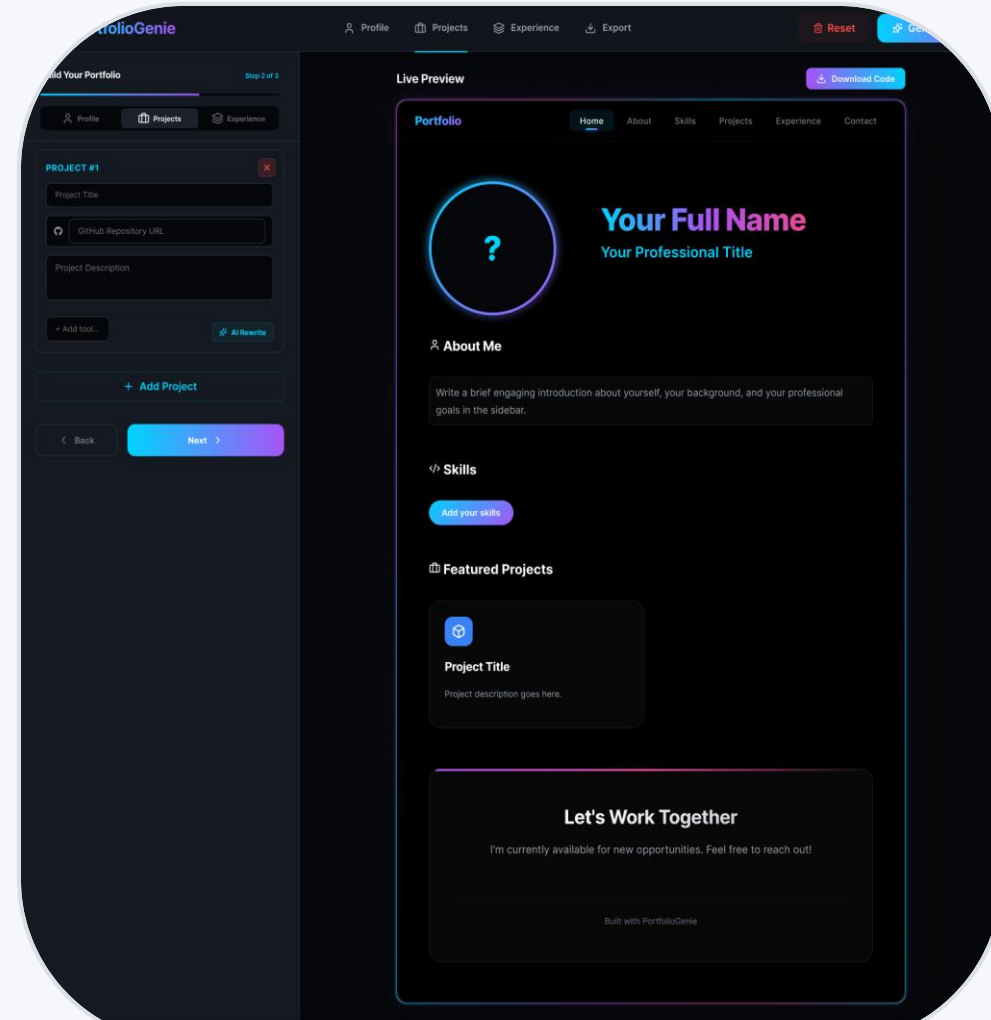
- ◆ Async API Queries: Custom hook (useGithub) managing async/await API connections.
- ◆ Profile Data Fetching: Fetches public profile details (bio, name, avatar, and repository list) from the `/users/{username}` endpoint.
- ◆ Parallel Request Resolving: Uses Promise.all to fetch language URLs for repositories concurrently.
- ◆ Error Boundary Fallbacks: Ensures the builder does not crash in case of network failures or incorrect usernames.

UML Sequence Diagram: GitHub Fetch Process in PortfolioGenie



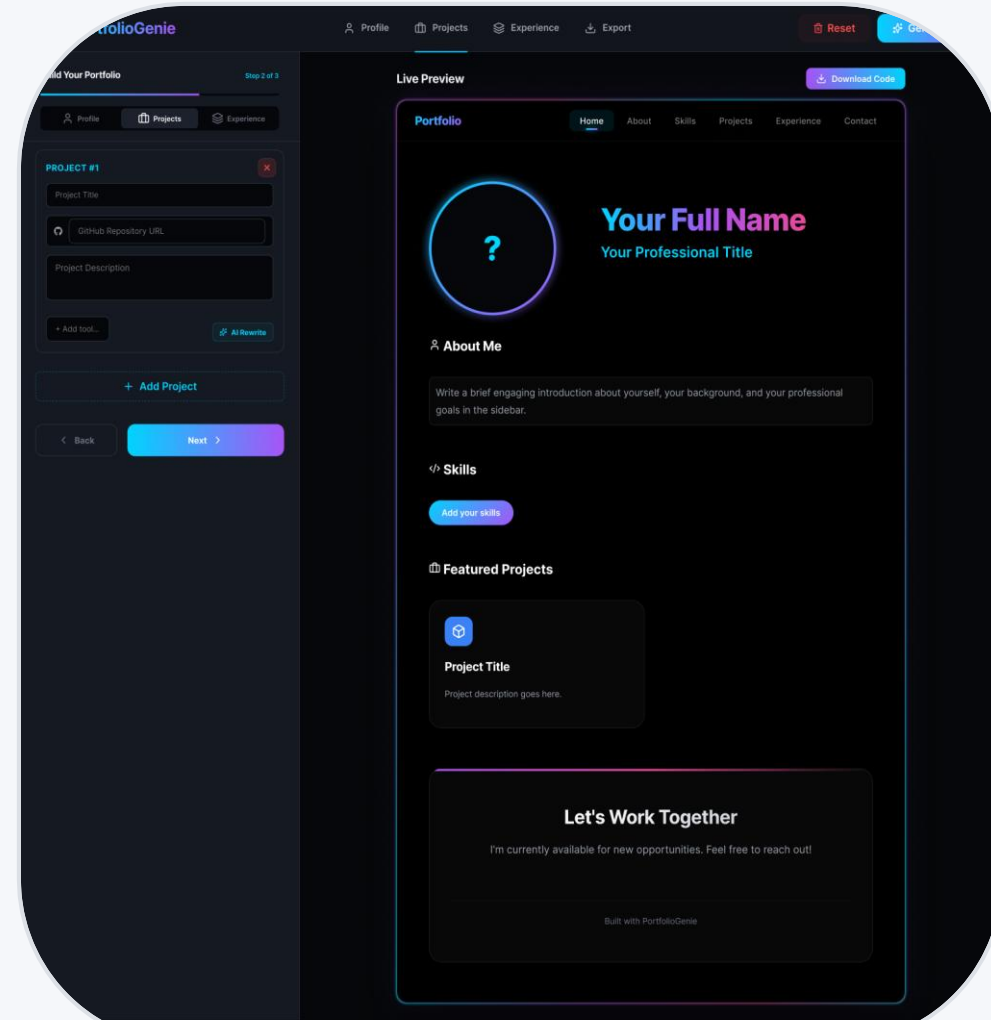
Data Formatting & Caching

- ◆ Languages Processing: Extracts unique language keys and merges them into the global skills state.
- ◆ Fallback Tag Settings: Appends ['Software'] as a default tag for repositories missing languages.
- ◆ Topics Parsing: Capitalizes repository topics and appends them as project tech tags.
- ◆ GitHub Rate Limit Cache: Saves fetched results in session state to stay under the 60 requests/hr limit.



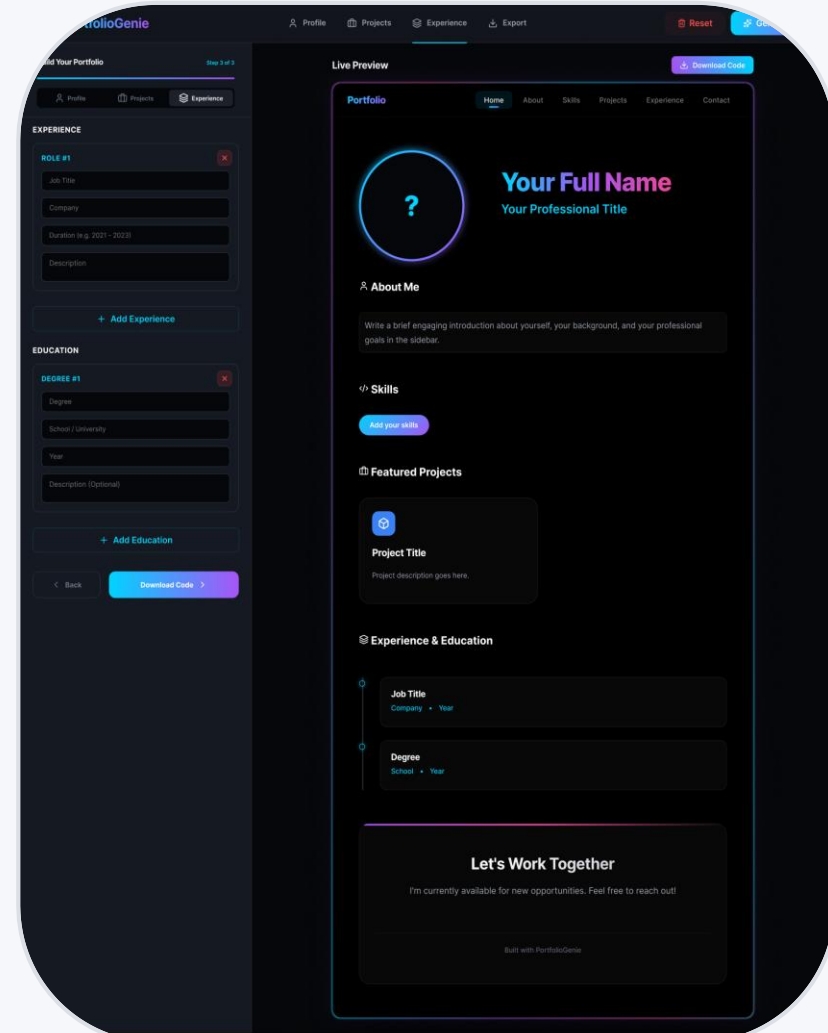
What I Built (ProjectsTab.jsx)

- ◆ Dynamic Cards Management: Maps projects from context state into cards with delete buttons.
- ◆ Project fields configuration: Text input controls for project title, GitHub repository URL, and description.
- ◆ Manual Project Creator: 'Add Project' action button that appends empty cards to showcase offline work.
- ◆ Project tag editor: Inline input field that appends custom tools to the tech tag array on Enter.



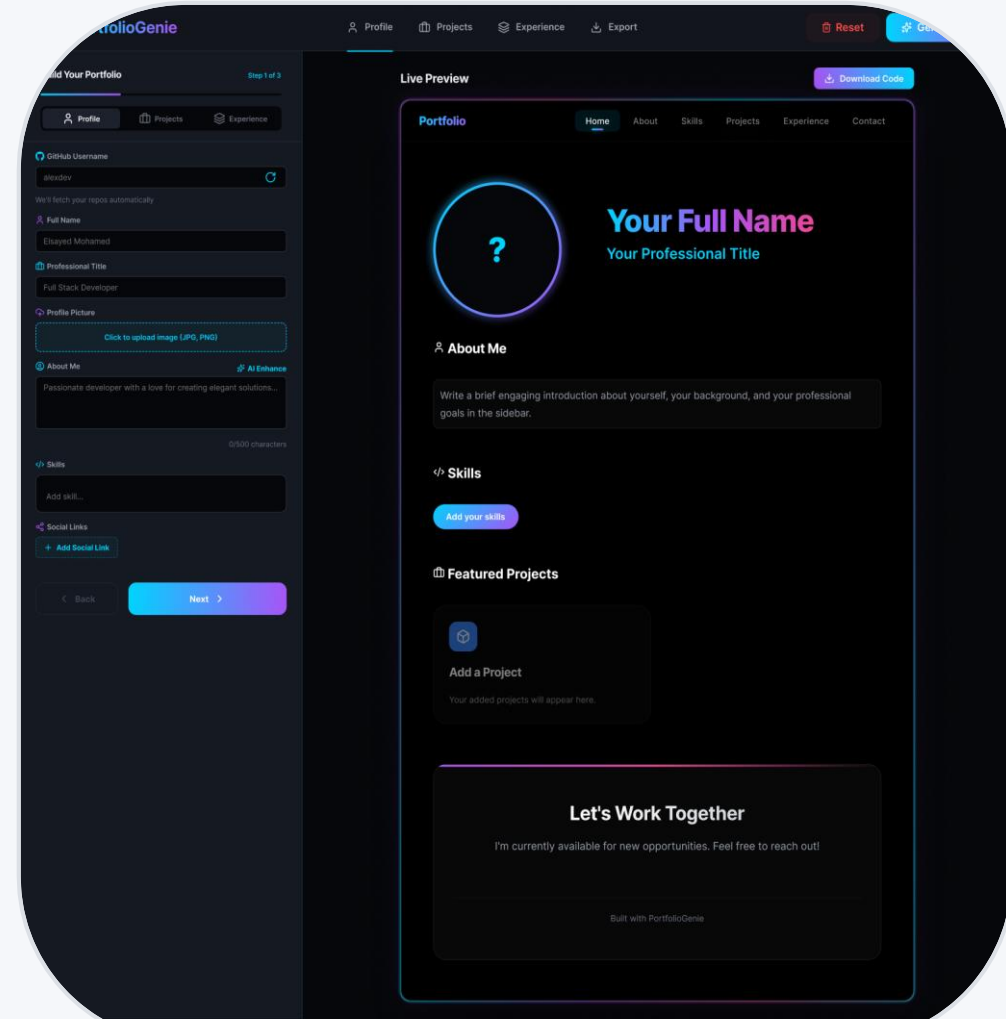
What I Built (ExperienceTab.jsx)

- ◆ Timeline Fields: Card configurations for Experience (role, company, year) and Education (degree, school).
- ◆ Dynamic List Modifiers: Context hook actions that append, edit, or delete items on the fly.
- ◆ Visual Connectors: Maps items into vertical list structures with CSS dots and timeline lines.
- ◆ Transition Animations: Smooth fade-in Framer Motion transitions (X offset -10 to 0) on component mount.



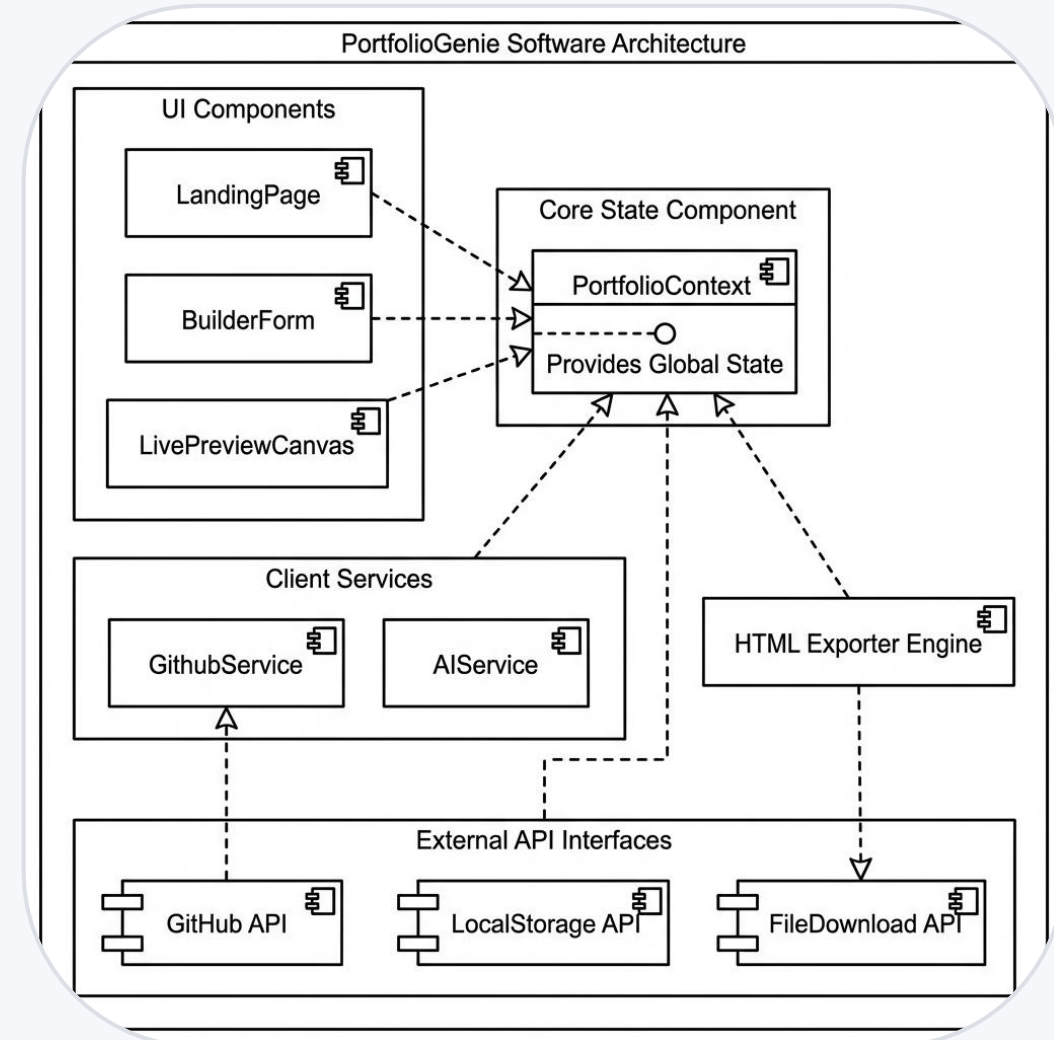
AI Copywriting Assistant

- ◆ Prompt generator utility: Formulates professional system prompt templates for bio and project text areas.
- ◆ Context interpolation: Injects user name, title, and key skill details into optimized sentences.
- ◆ Simulated AI Responses: Returns polished paragraphs with a 1.2s delay to simulate API response times.
- ◆ UI loading spinner state: Connects an `isGenerating` boolean state to render spinners on active buttons.



Export Engine & Context State

- ◆ DOM compiler utility: exportPortfolio.js clones active elements and inlines CSS styles.
- ◆ Media Queries parser: Filters keyframes, font-faces, and media queries to include in the output.
- ◆ prettifyHTML function: Formats HTML indentation before downloading the bundle.
- ◆ Context debouncer: Implements useReducer state with a 400ms setTimeout LocalStorage auto-save.



Project Milestones

- ◆ Developed a responsive visual portfolio builder that automates profile setup to under 5 minutes.
- ◆ Created a serverless deployment compiler that exports clean, single-file HTML code.
- ◆ Showcases track core competencies in React 18 component modeling and state management.

Future Improvements

- ◆ Integrate real LLM endpoints (Gemini API) for custom project and bio copywriting.
- ◆ Add multiple design layouts (dark mode themes, grid variants, vertical/horizontal timelines).
- ◆ Add automated CI/CD deployment pipelines directly to GitHub Pages and Netlify.

THANK YOU

PortfolioGenie